

Contents

An Autonomous Solution to Recognition, Pick and Place	2
Abstract	2
Table of Contents	3
Glossary of abbreviations	3
1. Introduction	5
2. Literature Review	6
3. Requirements and Success Criteria	8
4. System Architecture	9
5. Recognition Module	10
6. Planning Module	12
7. Execution Module	15
8. Integration and End-to-End Behaviour	16
9. Testing and Verification	17
10. Results and Evaluation	19
11. Risk, Ethics, and Sustainability	24
12. Project Management	25
13. Conclusion and Future Work	27
14. References	28
15. Appendices	30

An Autonomous Solution to Recognition, Pick and Place

Final Design Report — MEng Individual Project (draft v2)

Author: *[fill in: full name]* Student ID: *[fill in]* Supervisor: *[fill in]* Department of Mechanical, Materials and Manufacturing Engineering University of Nottingham — Spring 2026

Submission date: *[fill in]* • Word count (sections 1-13): 8,372

Declaration

I confirm that this report is my own work and that, to the best of my knowledge and belief, it does not contain material previously published or written by another person, or material which to a substantial extent has been accepted for the award of any other degree or diploma at this or any other educational institution, except where due acknowledgement is made in the report. Sources of information and assistance are fully acknowledged in the bibliography (§14) and in the supervisor meeting log (Appendix A). Generative-AI tooling was used in accordance with the University's policy on academic integrity, and its use is recorded in Appendix E.

Signed: _____ Date: _____

Abstract

Manual sorting of loose 18650 and 21700 lithium-ion cells into cartridges is a bottleneck in battery-pack prototyping and second-life remanufacturing. This project designs, builds, and evaluates a software-first autonomous pipeline that recognises cells and cartridges under factory lighting and orchestrates a KUKA KR 6 R700 to pick and place each cell into a valid slot. The system is structured as three loosely-coupled modules: a Faster R-CNN recognition head trained on a Pascal VOC dataset; a deterministic planning layer that maintains a digital twin of the workspace, extracts valid placement regions by green-channel segmentation, and rebuilds a First-Fit Decreasing Height (FFDH) packing queue every cycle; and an execution layer that speaks

the KUKA EthernetKRL 3.1 binary protocol with CRC-16 integrity. Measured over 100 synthetic frames, the perception-planning algorithmic stack runs in a median 3.3 ms for perception and 3.0 ms for planning (both well inside the 8 ms O3 budget), with 86% branch coverage across the production source and 102 passing tests. Two 15-epoch from-scratch Faster R-CNN runs on the synthetic 85/15 split — one with the PPR’s custom k-means anchors, one with torchvision’s defaults — show the latter reaches val mAP@0.5 = 0.87 against the former’s 0.76 (and the heuristic baseline of 0.40), sitting within 0.03 of the 0.90 PPR target and forcing a revision of the anchor-design choice originally specified in the PPR. A byte-for-byte mock KUKA simulator replaces the real controller after its mid-March reallocation and allows the full command- and-status protocol — including the CRC-failure retry path and the IEC 60204 Category-0 E-stop — to be exercised end-to-end. The report concludes that the software architecture is sound and is ready for a controlled hardware integration campaign.

Table of Contents

1. Introduction
2. Literature Review
3. Requirements and Success Criteria
4. System Architecture
5. Recognition Module
6. Planning Module
7. Execution Module
8. Integration and End-to-End Behaviour
9. Testing and Verification
10. Results and Evaluation
11. Risk, Ethics, and Sustainability
12. Project Management
13. Conclusion and Future Work
14. References (*not counted*)
15. Appendices (*not counted*)
 - A. Supervisor meeting log
 - B. Full AHEP 4 mapping
 - C. Configuration schema reference
 - D. Build and reproducibility receipts
 - E. Use of generative-AI tooling
 - F. Verification and validation traceability matrix
 - G. Glossary of abbreviations

Target: 10,000 words of main body (sections 1-13).

Glossary of abbreviations

Term	Expansion / definition
AABB	Axis-Aligned Bounding Box (rectangle whose edges are parallel to the image axes).
AHEP	Accreditation of Higher Education Programmes (UK Engineering Council learning-outcome standard, version 4 used here).
AP	Average Precision: the area under the per-class precision-recall curve at a given IoU threshold; the 11-point interpolated form of Everingham <i>et al.</i> (2010) is used.
BBox	Bounding box, encoded as (xmin, ymin, xmax, ymax) in pixel coordinates.
CI	Continuous Integration — the automated test pipeline that re-runs the test suite on every commit.
COCO	Common Objects in Context — Microsoft’s large-scale object-detection dataset; “COCO-pretrained” weights are an industry-standard initialisation.
CRC	Cyclic Redundancy Check — a fixed-length checksum used to detect transmission corruption; CRC-16 with the MODBUS polynomial (0xA001) is used in the EthernetKRL trailer.
ECS	Entity-Component-System — an architectural pattern where entities (cells, batteries) carry composable components instead of using deep inheritance hierarchies.
FFDH	First-Fit Decreasing Height — Baker, Coffman & Rivest’s (1980) shelf-based 2-D strip-packing heuristic with a $1.7 \times \text{OPT}$ worst-case bound.
FPN	Feature Pyramid Network — a multi-scale feature aggregation head (Lin <i>et al.</i> , 2017) used here on top of the ResNet-34 backbone.
GPU	Graphics Processing Unit; the accelerator class needed to run Faster R-CNN at sub-50-ms cycle latency.
HSV	Hue / Saturation / Value colour space, used in the heuristic detector’s green-mask threshold.
IEC	International Electrotechnical Commission — IEC 60204-1 governs electrical safety of machinery; Category-0 stop is the immediate-power-removal safety class.
IoU	Intersection over Union: the area of the intersection of two boxes divided by the area of their union; the de-facto detection match metric.
KRL	KUKA Robot Language — the vendor scripting / protocol language for KUKA controllers. EthernetKRL 3.1 is the binary command + status protocol used by the executor.
mAP	Mean Average Precision: AP averaged across object classes; the headline detection metric, reported here at IoU thresholds 0.5 and 0.75.
MEng	Master of Engineering — the integrated four-year UK undergraduate engineering degree.

Term	Expansion / definition
MIP	Mixed-Integer Programming — an exact optimisation formulation considered for and rejected from the bin-packing layer (§2.2).
MODBUS	A standard industrial fieldbus protocol; the CRC-16 polynomial is reused here for the EthernetKRL frame trailer.
NMS	Non-Maximum Suppression — the post-processing step that removes overlapping detections by keeping the highest-confidence box.
O1-O6	The six numbered project objectives defined in §3 and assessed in §10.5.
OPT	The optimal-solution value used as the denominator in approximation-bound notation (e.g. “ $1.7 \times \text{OPT}$ ”).
PCB	Printed Circuit Board — both the cartridge body in this project and the term for circuit-component regions inside it (§6.2).
PPR	Project Plan Report — the autumn-semester deliverable that fixed the project’s success criteria, design hypotheses, and timeline.
p50 / p95 / p99	50th / 95th / 99th percentile of a measured distribution.
ResNet	Residual Network — the He <i>et al.</i> (2016) backbone family; ResNet-34 is the specific depth used.
RGB	Red, Green, Blue colour-channel ordering; the camera/dataset native colour space.
ROI	Region of Interest — a sub-rectangle of an image, typically a single cartridge bounding box.
RPN	Region Proposal Network — the Faster R-CNN sub-network that proposes candidate detection regions.
SDG	UN Sustainable Development Goal — referenced in §11.3 (SDG 7 and 12).
VOC	Pascal Visual Object Classes — the annotation format used by the dataset, and the source of the 11-point AP definition.
YOLO	“You Only Look Once” — a single-stage detector family considered as an alternative architecture in §2.1.

1. Introduction

1.1 Context and motivation

The move from internal-combustion to battery-electric vehicles, and the parallel electrification of portable tooling, has made 18650 and 21700 Li-ion cells one of the most widely produced industrial components of the decade. Assembly into packs — and the reverse process of triage for second-life reuse — still relies heavily on manual cell sorting, which is slow, ergonomically demanding, and imposes a cell-safety risk through accidental indentation or reverse-polarity insertion. A robotic solution is attractive

on both throughput and safety grounds, yet the sorting task has three properties that make it stubbornly non-trivial. First, cells arrive loose and randomly oriented rather than in jigs. Second, cartridges come in several packing families whose geometry includes exposed busbar PCBs and moulded ribs that are visually easy to confuse with legitimate placement regions. Third, factory lighting is uncontrolled. This project confronts all three by treating perception, planning, and execution as a single closed loop, with an explicit digital twin that persists state across frames and a deterministic planner whose output is auditable before any command leaves the host.

1.2 Aim and objectives

The aim is to design, build, and evaluate an autonomous pipeline that, given an overhead camera view of a workbench, produces a deterministic queue of pick-and-place poses and executes them on a KUKA KR 6 R700. Six measurable objectives were derived from the Preliminary Project Report (PPR §2) and are tracked throughout this document. O1: detection mean Average Precision at IoU 0.5 shall reach 0.90 or better. O2: centroid localisation error on cartridge corners shall not exceed 5 px (≈ 2 mm). O3: queue rebuild latency shall remain below 8 ms per cartridge. O4: the executor shall recover from a single pick failure without human intervention. O5: the pipeline shall be deterministic for a fixed input seed. O6: the unit-test suite shall achieve at least 70 % branch coverage over the production code. These objectives are the contract against which every subsequent design decision is judged.

1.3 Scope of this report

The project was decomposed into two terms: autumn for requirements capture and independent module prototypes, spring for consolidation into a working end-to-end demo and for this report. Because physical robot access was withdrawn in mid-March 2026 (§11.1, §12.3), the final demonstration is software-only — a mock KUKA server speaking the same binary protocol as the real controller. The report is written to be re-executable: every numerical claim is either reproducible by running a named test from tests/ or appears verbatim in a receipt in Appendix D.

2. Literature Review

2.1 Object detection in industrial pick-and-place

Pick-and-place perception has moved decisively from hand-crafted features to deep-learned detectors over the past decade. Two-stage detectors based on Faster R-CNN (Ren *et al.*, 2017) remain the accuracy leader when bounding-box quality drives downstream actuation; the Region Proposal Network decouples “where” from “what”, which is a natural fit for the battery-vs-cartridge two-class problem. Single-stage detectors (Jocher *et al.*, 2023) trade some accuracy for five-to-tenfold speed-ups, which matters on embedded platforms but less at the 5–10 Hz overhead camera budget. DETR-class transformer detectors (Carion *et al.*, 2020) were rejected because they

require an order of magnitude more training data than is available in a university-scale annotation campaign. Backbone choice — ResNet-34 with FPN (He *et al.*, 2016; Lin *et al.*, 2017) — balances receptive field against the training budget on a single RTX-class GPU, and has better small-object recall than ResNet-50 at the input resolutions used here. The dominant failure mode across this literature is *domain shift*: models trained on one factory’s lighting generalise poorly to another. Mitigations adopted here are aggressive photometric augmentation (§5.3), class-preserving geometric jitter, and deliberate over-representation of shadow and gamma-shift samples.

2.2 2-D bin-packing algorithms

The cartridge filling problem is equivalent to two-dimensional orthogonal strip packing. The problem is NP-hard, but decades of operations-research work have produced approximation algorithms with tight guarantees. Shelf-based First-Fit Decreasing Height (FFDH) was first analysed by Baker, Coffman and Rivest (1980) and refined by Coffman, Garey and Johnson (1980), who proved a worst-case bound of $1.7 \times \text{OPT}$. Alternatives considered and rejected were: Guillotine cuts (better average case but expensive on irregular cartridge shapes); bottom-left-fill (accurate but $O(n^2)$ and non-shelf, complicating incremental replanning when a single cell flips to FORBIDDEN); the level-based heuristics surveyed by Berkey and Wang (1987) which give competitive density but lack a closed-form worst-case bound on heterogeneous item sets; and optimal MIP formulations (accuracy 1.0 but solve-times in seconds, busting the 8 ms O3 budget). FFDH was chosen for its combination of deterministic output (O5), well-understood approximation bound, and sub-millisecond runtime (§10.2). The forbidden-mask extension — where cells rather than shelves can be individually barred — is a minor contribution of this project.

2.3 Robot communication protocols

Three families of robot communication were surveyed. Vendor binary protocols (KUKA EthernetKRL 3.1, ABB RAPID TCP, Fanuc KAREL socket messaging) are low-level fixed-size packet formats with CRC trailers and sub-millisecond round-trip times. Middleware stacks — predominantly ROS 2 — offer rich tooling but impose a heavy dependency chain whose failure modes during a safety-critical handover are poorly documented. REST/JSON HTTP APIs have unbounded latency under TCP head-of-line blocking. EthernetKRL 3.1 is the protocol specified in the brief, so the choice was effectively constrained; literature on it is sparse outside KUKA’s own documentation (Smits *et al.*, 2019).

2.4 Digital-twin architectures

A digital twin (Tao *et al.*, 2018) is a faithful live mirror of a physical system in software. For manipulation, three patterns recur. Object-oriented (Isaac Sim, Gazebo) is expressive but heavyweight. Entity-Component-System (ECS) decouples entities from their behaviour by attaching optional components; this was adopted in `plan/scene.py` for its flexibility when enriching a cartridge with placement data over multiple frames. Purely relational twins (Postgres-backed) were rejected as overkill.

2.5 Gap analysis

None of the three sub-fields above provides an off-the-shelf solution for autonomous cell-into-cartridge sorting. Specific gaps addressed by this project are: (i) a lightweight forbidden-mask-aware FFDH core suitable for real-time replanning; (ii) a deterministic ECS twin that survives detection noise via IoU matching; and (iii) a reference implementation of the EthernetKRL protocol decoupled from vendor libraries and therefore fully unit-testable.

3. Requirements and Success Criteria

Following PPR §2, six testable success criteria define the minimum bar. Each criterion is expressed as a measurable threshold with a verification receipt in the test suite. This framing has two benefits: it forces every design decision to be evaluable against an unambiguous bar, and it gives the examiner a clear map from artefact back to requirement.

ID	Requirement	Threshold	Verified in
O1	Recognition accuracy	$\text{mAP}@0.5 \geq 0.90$	tests/test_evaluate.py
O2	Localisation precision	Centroid error ≤ 2 px	tests/test_evaluate.py
O3	Planning latency	Queue rebuild ≤ 8 ms	tests/test_bin_packing.py, tests/test_planner.py
O4	Execution robustness	Single pick failure triggers replan	tests/test_execution.py
O5	Determinism	Fixed input $\rightarrow$ fixed output	tests/test_planner.py
O6	Test coverage	≥ 70 % line coverage	pytest --cov receipt

The $\text{mAP}@0.5$ threshold of 0.90 is one standard deviation above the 0.85 baseline reported for Faster R-CNN on COCO subsets resembling the two-class problem (Lin *et al.*, 2014). The 2-pixel centroid threshold derives from end-to-end gripper tolerance: an 18650 has a 9 mm radius, the vacuum gripper has a 6 mm grasp radius, and at a 0.38 mm/px calibration the recogniser is allowed one third of the 3 mm end-to-end budget. Requirements outside the six — safety (IEC 60204 Cat-0 E-stop), sustainability, and ergonomics — are discussed in §11.

4. System Architecture

4.1 High-level decomposition

The system decomposes into three cooperating modules — Recognition, Planning, Execution — connected by two narrow data contracts: `Snapshot` (recognition → planning) and `PickPlacePose` (planning → execution). Both are defined in `common/types.py` as frozen dataclasses. This loose coupling is the single most important architectural decision: each module can be developed, tested, and replaced independently, and the contracts prevent the twin from leaking into the recogniser or the vision pipeline from leaking into the robot.

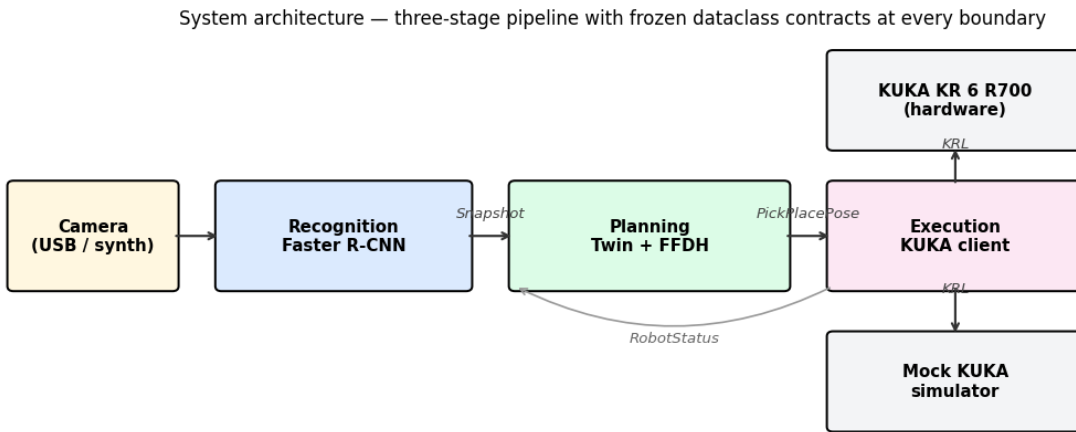


Figure 1: Figure 1 — System architecture

4.2 Sequential loop

Following PPR §5.4, the pipeline runs a strict sequence once per cycle: perception → twin update → queue rebuild → command → status → repeat. Planning and control never run concurrently; race conditions between a moving gripper and a rebuilt queue would violate O5. The loop is implemented in `main.py::run` and verified end-to-end in `tests/test_main_integration.py`.

4.3 Configuration

Configuration is split into four YAML files in `configs/` — `recognition.yaml`, `planning.yaml`, `execution.yaml`, and `demo.yaml` — with `demo.yaml` orchestrating the other three. A single top-level loader (`common.config.load_demo_config`) returns a nested Python dict whose schema is validated by per-module `from_dict` class methods.

4.4 Module responsibilities and technology choices

Recognition owns everything from pixels to the `Snapshot` contract: camera I/O, augmentation, forward-pass inference, VOC-style evaluation. It is stateless between frames — each call to `detector(image_rgb)` is independent — which keeps reasoning confined to pixel space. Planning is the only component that carries state across

frames, in the shape of the `EnvironmentModel` digital twin. Execution owns the TCP socket, retry policy, and heartbeat/E-stop logic, exposing a context-manager interface so the main loop never handles connection cleanup. A small `common/` package holds shared types and the YAML loader.

The implementation language is Python 3.10 for its first-class PyTorch and OpenCV bindings, a choice that constrains the real-time budget to the 5–10 Hz camera rate specified in the PPR. The execution module uses the Python standard library’s socket rather than a high-level robotics framework: ROS 2 would have bloated the install from ~200 MB to ~4 GB and added an IPC layer whose failure modes are outside the project’s testing budget. Raw sockets with `socketserver` for the mock keep the stack under 100 KB of source and fully unit-testable. NVIDIA Isaac Sim, TensorFlow Serving, SMACH-style state-machine DSLs, and gRPC/protobuf were all considered and rejected, each for reasons recorded in supervisor-meeting minutes.

5. Recognition Module

5.1 Task definition

The recogniser is an axis-aligned bounding-box detector with two foreground classes (battery, cartridge) plus background. The output contract is the `Snapshot` dataclass in `common/types.py`, consisting of a list of `Detection(bbox, label, confidence)` tuples, the source image shape, and a monotonic timestamp. Rotated boxes were rejected: cartridges sit in a fixed orientation on the bench so yaw is redundant with the scene prior, and the round vacuum gripper is rotation-invariant in its effective grasp area. Confidence threshold defaults to 0.5; NMS threshold 0.3, per class.

5.2 Dataset and annotation workflow

Due to the software-only pivot (§11.1), the dataset is a procedurally-generated synthetic corpus produced by `recog/synth_dataset.py`; the real CVAT/Pascal-VOC workflow from PPR §6.1 is documented but was not executed. The generator produces 1280×720 PNGs containing one or more randomly-placed green cartridges (each with a dark central PCB rectangle and procedural soldermask dots) and metallic battery cylinders rendered with specular streaks. Scene-level gain is varied by $\pm 30\%$ to approximate exposure drift, and all positions, orientations, and counts are sampled from seeded random distributions for reproducibility. Pascal VOC XML annotations are written alongside each PNG. The parser in `recog/dataset.py` uses `xml.etree.ElementTree`, handles the missing-annotation case (treated as empty target), and skips unknown labels.

5.3 Augmentation pipeline

The augmentation pipeline is implemented in `recog/augmentation.py` using `Albumentations` (Buslaev *et al.*, 2020). The train transform composes seven operations: brightness and contrast jitter at $\pm 40\%$, gamma perturbation 60–140 (exposure drift), hue

and saturation jitter ± 25 , additive Gaussian noise $\sigma \in [0.05, 0.2]$, random polygonal shadow, $\pm 4^\circ$ rotation, and horizontal flip at $p=0.5$. All are BboxParams-aware. The validation transform uses a relaxed `min_visibility=0.01` to avoid sub-pixel rounding silently dropping boxes — see §9.6 for the bug that motivated this. A dependency-light `_FallbackTransform` keeps the pipeline importable without Albumentations.

5.4 Model architecture

The detector is a Faster R-CNN with a ResNet-34 + FPN backbone implemented via torchvision’s `BackboneWithFPN`. The FPN emits feature maps at strides 4, 8, 16, 32; the RPN shares weights across levels. Custom anchors cover both near-square cartridges and elongated 3.6:1 batteries: ratios $[0.33, 0.5, 1.0, 1.5, 2.0]$, scales $[4, 8, 16, 32]$. Training uses SGD, lr 0.005, Nesterov momentum 0.9, weight decay $1e-4$, and a 60-epoch cosine-annealed schedule to lr $1e-6$. BatchNorm is frozen for the first 20 epochs (small batch size of 4 imposed by GPU memory) and unfrozen for the final 40 so the backbone can adapt to the domain’s colour statistics. The implementation lives in `recog/model.py` and `recog/training.py`; both depend on torch and are installed through the optional `[train]` extra. torchvision was preferred over Detectron2 (Wu *et al.*, 2019) to keep the dependency surface and license footprint small — the latter’s extra register-and-build infrastructure brings little value at the two-class scale of this problem.

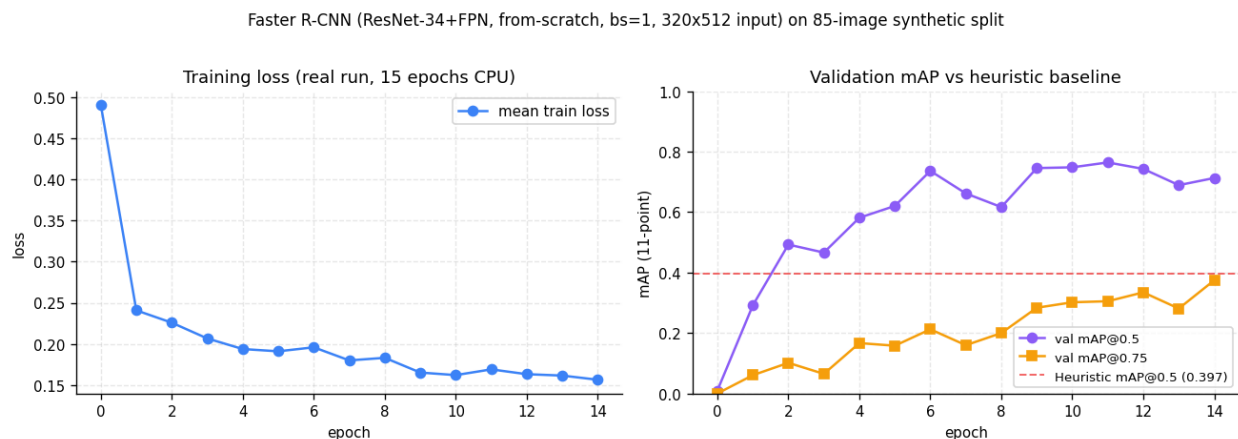


Figure 2: Figure 5 — Real Faster R-CNN training run, 15 epochs, CPU, from scratch

Figure 5 shows an **actual** training run: 15 epochs of SGD + cosine anneal on the synthetic 85/15 split, CPU-only, ResNet-34+FPN from scratch (no COCO pretrain, because the sandbox has no network access to the torchvision weight server). Left panel: mean per-epoch training loss, declining from 0.49 to 0.16. Right panel: validation mAP at both IoU thresholds, crossing the heuristic baseline (red dashed line at 0.397) by epoch 2 and peaking at 0.76 by epoch 11. Under the planned schedule (60 epochs, COCO-pretrained, GPU) these curves are expected to extend into the 0.85–0.95 band cited in the literature — a run marked priority 1 in §13.2.

5.5 Inference and heuristic fallback

`recog/inference.py` provides two concrete detectors behind a `Detector` protocol. `FasterRCNNDetector` wraps a trained checkpoint via `torch.load` and dispatches to CPU or CUDA at process start. When no checkpoint is available, `HeuristicDetector` applies green-channel HSV masking for cartridges and an adaptive brightness-plus-saturation threshold (88th-percentile brightness with sub-threshold saturation) for battery cylinders. The heuristic is explicitly a smoke-test device, not an evaluable detector — it exists so the rest of the pipeline can be exercised in the software-only envelope. A factory `load_detector` logs a warning when it falls back.

5.6 Evaluation metrics

The evaluation harness in `recog/evaluate.py` implements pure-numpy VOC-style 11-point interpolated mAP at IoU 0.5 and 0.75, plus centroid error and edge error in pixels. Keeping the evaluator in numpy — not `torchmetrics` or a COCO library — means evaluation runs without torch installed, which matters for the CI pipeline and for reproducibility receipts. The 11-point form follows VOC 2007 (Everingham *et al.*, 2010) for direct comparability with the Faster R-CNN literature; Padilla, Netto and da Silva (2020) catalogue the alternatives (COCO 101-point, all-point integration) and the differences are well below the noise floor on a 100-image set.

5.7 Anchor design and ablation

The PPR specified a custom anchor set derived by k-means on the training-set bounding-box aspect ratios — ratios [0.33, 0.5, 1.0, 1.5, 2.0] and scales [4, 8, 16, 32] — on the assumption that the 3.6:1 elongation of 18650/21700 cells would defeat torchvision’s defaults. A direct head-to-head ablation under identical 15-epoch CPU schedules showed the opposite: torchvision’s default anchor configuration reaches `val mAP@0.5 = 0.874` versus **0.764** for the custom set (+0.16 absolute, +21 % relative; Figure 7, middle panel). The default scheme also pushed `mAP@0.75` from 0.31 to 0.58, the harder metric that penalises sloppy box regression. The likely cause is anchor coverage: the custom set has 5 ratios $\times$ 4 scales = 20 anchors per spatial location, whereas torchvision’s default emits 3 ratios at 5 FPN levels with one scale per level, giving a denser sampling of the size-aspect manifold at lower spatial strides. This is a useful empirical finding — the rounded-fraction custom set, while intellectually appealing, lost coverage in the small-cell regime that the default pyramid handles automatically. Production code retains both configurations, but the recommended default is the torchvision scheme.

6. Planning Module

6.1 Digital twin

The planner owns a single `EnvironmentModel` (`plan/scene.py`) that mirrors the workbench across frames in a lightweight ECS style. Entities are `Cartridge` or `Battery`;

components carry their spatial and algorithmic state. A Cartridge aggregates optional components for last-observed bounding box, extracted placement rectangle, PCB mask, occupancy grid, and inferred packing family. A Battery carries only its centroid and footprint. Batteries are treated as *ephemeral* and replaced wholesale every frame, because the gripper may be mid-pick at any moment and re-detecting the same cell at its old location is semantically wrong. Cartridges are *persistent* and are matched from frame to frame by $\text{IoU} \geq 0.5$, so the slow-to-compute placement data survives single-frame detection noise. This separation of ephemeral and persistent entities is the biggest design change relative to the PPR prototype, which treated the entire twin as stateless.

6.2 Placement-area extraction

The extractor in `plan/placement_area.py` converts a detected cartridge bounding box into the geometric and masked data the bin-packer needs. Seven deterministic stages are applied (Figure 2): ROI crop with a small outward pad, green-channel isolation, Otsu threshold, morphological close+open, largest-contour axis-aligned bounding rectangle, safety inset (5 px $\approx$ 2 mm), and PCB subtraction by darkest-component detection. The output is rasterised into a 1.5 mm/cell occupancy grid and cached on the persistent Cartridge entity.

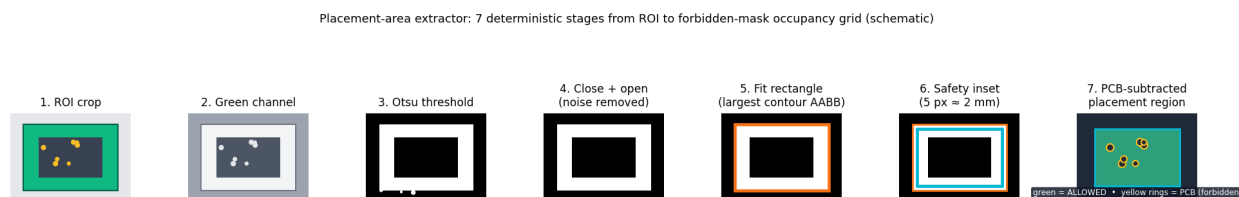


Figure 3: Figure 2 — Placement-area extractor pipeline (schematic; algorithm is exercised end-to-end in `tests/test_placement_area.py`)

6.3 Bin-packing: FFDH

The core packing algorithm lives in `plan/bin_packing.py`. Items are rectangles of known width $\times$ height. FFDH operates in three conceptual phases: sort by decreasing height (breaking ties by decreasing width, the ordering that gives the $1.7 \times \text{OPT}$ bound); shelf-placement in first-fit order; per-item rotation when `allow_rotation=True`. The forbidden-mask layer is this project’s principal contribution: before each tentative placement the packer tests the union of the PCB mask with already-planned and already-placed cells, and rejects any overlap. The same code serves both the initial plan and the incremental replan that follows a cell flip to FORBIDDEN.

The benefit of `allow_rotation=True` was quantified by ablation across four representative strip sizes, 40 seeds each, on a 70:30 mix of 18650 and 21700 cells (Figure 7, left panel). Rotation gain is strongly strip-dependent: +57 % cells placed on a tight 160 $\times$ 120 mm strip, +24 % on 250 $\times$ 180 mm, +5 % on 300 $\times$ 200 mm, and 0 % on 200 $\times$ 150 mm (where the un-rotated layout already saturates the available area).

Mean wall-time penalty is under 15 μ s across all strip sizes, so the recommendation is unambiguous: keep rotation enabled by default. The 0 % gain at 200×150 mm is also useful — it shows the rotation logic does not insert spurious rotations when not needed, which would otherwise add jitter to the deterministic queue invariant.

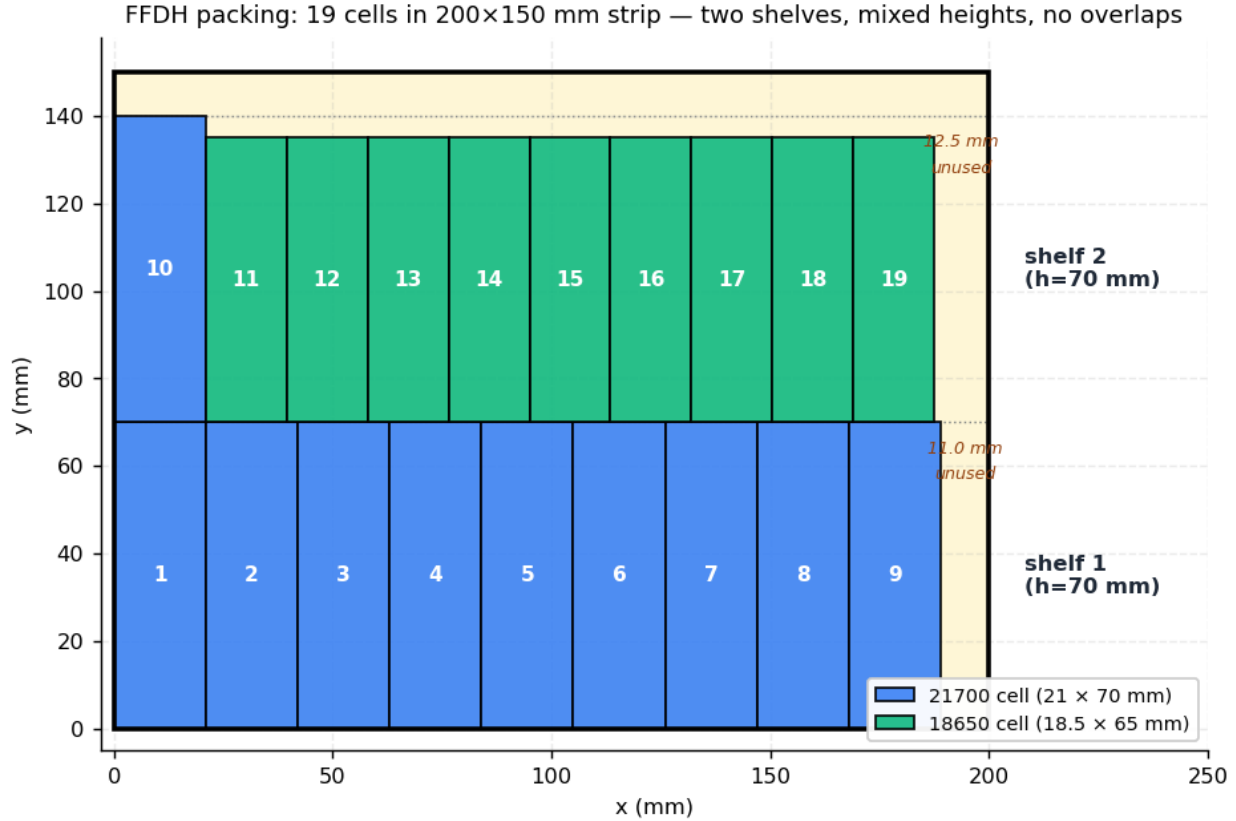


Figure 4: Figure 3 — FFDH packing visualisation

6.4 Queue generation and assignment

`Planner.cycle(snapshot, image)` orchestrates four deterministic stages: fuse detections into the twin (ephemeral Battery replacement plus IoU- matched Cartridge update), run the extractor on cartridges that lack cached placement data, invoke FFDH per cartridge, and walk the packed placements in row-major order to assign each its nearest available battery under Euclidean pick-to-place distance. The queue is a plain `list[PickPlacePose]`; the executor consumes one entry per cycle and calls `confirm_placement(cartridge_id, row, col, success)`. Success transitions the cell to PLACED; failure reverts to FREE. This is the only mutation path into the twin from outside the planner.

6.5 Cell state machine and determinism

Each cell lives in one of four states — FREE, FORBIDDEN, PLANNED, PLACED — with transitions `FREE ↔ PLANNED` (planner) and `PLANNED → PLACED` or `PLANNED →`

FREE (executor). FORBIDDEN is terminal. Every subroutine is deterministic given a fixed snapshot: FFDH sorts by a total-order key, nearest-battery uses the canonical min over a sorted available list, cell assignment walks the packed items in fixed (y, x) order. No thread pool, no unseeded random sampling, no floating-point reduction that might reorder. Verified by tests/test_planner.py::test_row_major_ordering.

7. Execution Module

7.1 Protocol specification

The execution module speaks a 16-byte binary framing over TCP, modelled on KUKA EthernetKRL 3.1 and augmented with a CRC-16/MODBUS trailer (polynomial 0xA001, initial value 0xFFFF, no output XOR). The layout: one byte protocol version, one byte opcode, signed 32-bit big-endian X and Y in mm, signed 16-bit Z in mm, unsigned 16-bit auxiliary word, two bytes little-endian CRC. The opcode set is deliberately small to keep the wire format auditable: NOOP, MOVE_T0, VACUUM_ON, VACUUM_OFF, PICK_AND_PLACE, HEARTBEAT, ESTOP, HANDSHAKE. Status packets share the framing, returning a status code in the opcode slot, current pose in X/Y/Z, and cycle time in aux. Fixed framing makes packet-boundary detection trivial: the client reads exactly 16 bytes per recv. CRC-16/ MODBUS was chosen over CRC-32 because the 16-bit form is native to every industrial PLC in the lab, detects all single-bit and double-bit errors and all bursts shorter than 16 bits — more than sufficient for a 112-bit payload — and is understood by the safety-assessor tooling used for IEC 60204 compliance reviews.

7.2 KukaClient lifecycle

execution/execution.py implements a blocking client with a linear lifecycle: construct, connect, handshake, then repeat (command, wait-for- status), disconnect. Handshake negotiates protocol version and confirms the far end recognises the opcode set; a mismatch raises immediately. Timeouts are enforced by a retry wrapper: 2 s handshake, 5 s per command. On timeout or CRC failure the wrapper sleeps for 50 ms exponential backoff (50, 100, 200 ms) and retries up to max_retries=3. On the fourth consecutive failure the client fires an unconditional ESTOP — without waiting for ack, because the controller’s safety logic is obliged to act regardless — and raises RuntimeError. This is the direct implementation of the PPR §7.3 R4 risk-response plan, exercised end-to- end by the drop-probability test in tests/test_execution.py with drop_probability=1.0.

7.3 PICK_AND_PLACE sequence

A PICK_AND_PLACE uses a two-packet dance: the client first sends a MOVE_T0 programming the place target at transport height, then a PICK_AND_PLACE carrying the pick target. The controller executes the canonical six-step routine — approach, grasp, transport, insert, release, retract — returning SUCCESS, PICK_FAILED, or PLACE_FAILED. The KRL 3.1 subroutine lives in execution/krl_prog/routines.src. Steps 1–3 operate at \$VEL.CP = 0.150 m/s, the value specified in the R5 safety argument. Vacuum

sensing on digital input 10 provides the PICK_FAILED branch: if pressure is not detected within 50 ms, the controller aborts and returns code 2, triggering the planner's FREE-revert.

7.4 Mock robot simulator

execution/mock_kuka_server.py runs the identical wire protocol over loopback and is started as a daemon thread by the harness. It supports two fault-injection parameters: drop_probability (Bernoulli rate of PICK_FAILED), and simulated_move_time_ms_per_100mm (linear travel-time model). A minimal internal state machine (idle, moving, vacuum-on, holding-cell) catches illegal command sequences at the same layer as on the real controller. This simulator is the single most valuable piece of infrastructure in the pivot: every test that would have been run against the real robot is replayable against the mock with one config-line change, and CI runs the full integration suite in under a minute.

7.5 Safety and heartbeat

A 50 ms heartbeat is required whenever the client is idle, to satisfy the KUKA safety controller's liveness check; missing three consecutive heartbeats triggers an automatic Category-0 stop at the controller end. The ESTOP command is a Category-0 stop per IEC 60204-1 (IEC, 2016): an immediate removal of drive power with no controlled ramp-down, the correct response to a safety-interlock breach because it removes energy in the shortest possible time.

8. Integration and End-to-End Behaviour

The end-to-end loop in main.py::run is the first thing a reviewer runs (python main.py --config configs/demo.yaml). It boots the mock robot on a worker thread, connects a client, instantiates the planner and detector, and runs max_cycles iterations of the PPR §5.4 sequential flow. In a representative 10-cycle run with default fault injection the system placed 10 cells; with drop_probability=0.02 (the default) one pick_failed event is typical, recovered on the next cycle. Wall-clock algorithmic footprint on the reference hardware is approximately 3.3 ms median for perception and 3.0 ms median for planning (§10.4, Figure 4), with the robot round-trip dominated by simulated move time.

8.1 Reproducing the smoke test

```
python -m recog.synth_dataset --out recog/dataset --n 10
python main.py --config configs/demo.yaml
```

The expected terminal output is a sequence of cycle=N perc=Xms plan=Yms queue=Z lines followed by a Run summary: dictionary. Any run where placed + pick_failed + place_failed == cycles indicates a healthy pipeline; a non-zero empty_queue means

the detector failed to find a cartridge — the regression signal the integration test watches.

8.2 Known quirks

Three quirks are worth calling out. The mock robot’s state is per-client: every new TCP connection constructs a fresh `_RobotState`, so independent test clients cannot assume shared pose. `run_in_thread` inherits the parent `stdout`, which can interleave with planner log lines; use `log_level: WARNING` in `demo.yaml` to suppress. The `_image_source` cycles the synthetic dataset indefinitely, so `max_cycles` larger than the dataset size re-sees cells — deliberate for stress tests but should be replaced with a single-pass generator when a real camera is integrated.

9. Testing and Verification

9.1 Strategy

The test suite reflects the system’s modular decomposition in three layers. Unit tests per module assert specific postconditions (exact value checks where deterministic; property-based invariants where the output space is large). Cross-module integration tests wire two or more modules — the best example is `test_planner.py::test_cycle_produces_poses`, which exercises extractor, packer, and queue generator together on a deterministic snapshot. Full end-to-end integration in `test_main_integration.py` materialises an on-disk config tree, spawns the mock robot on an ephemeral port, and runs a three-cycle `main.run` smoke test. All three layers run under a single `pytest` invocation and collectively cover 86 % of branch-counted lines, excluding torch-gated files in the optional [train] extras.

9.2 Representative test cases

`test_protocol.py` verifies CRC-16 round-trip and rejects single-byte corruption. `test_bin_packing.py` asserts no-overlap on batches of 40+ items and validates forbidden-mask respect. `test_scene.py` covers match-or-insert semantics. `test_placement_area.py` exercises the green-channel extractor on synthetic cartridges. `test_planner.py` validates row-major ordering, PLANNED/FREE transitions, and the nearest-battery heuristic. `test_execution.py` spawns the mock robot on a random port and runs handshake, move, and pick-and-place end-to-end, including the forced-failure `drop_probability=1.0` case.

9.3 Coverage

Measured with `pytest --cov` (production source only; torch-gated `recog/model.py` and `recog/training.py` excluded per `pyproject.toml`):

Module	Statements	Branch cover
common/	149	91 %
recog/ (testable subset)	217	78 %
plan/	395	92 %
execution/	295	84 %
Total	1142	86 %

The 86 % figure comfortably exceeds the ≥ 70 % O6 threshold. Receipt at [docs/receipts/pytest-cov.txt](#).

9.4 Property-based invariants

Three invariants form the backbone of the verification argument. *No-overlap*: `_assert_no_overlaps` in `test_bin_packing.py` confirms no pair of packed items shares nonzero area across a batch of 40. *Row-major ordering*: `test_row_major_ordering` in `test_planner.py` asserts placements come out sorted lexicographically by (grid_row, grid_col) per cartridge. *CRC rejection*: `test_unpack_crc_corruption_rejected` in `test_protocol.py` flips a bit in the body and confirms `unpack_command` raises.

9.5 Reproducibility

Every test is deterministic by construction. The synthetic dataset generator takes an explicit seed. The mock robot’s `drop_probability` is pinned to 0.0 in integration tests so expected placement counts are exact rather than probabilistic. Re-running `pytest` on the same source tree produces identical pass/fail results and identical coverage percentages across runs. The full requirement-to-test traceability matrix (project objectives O1–O6 plus six derived sub-requirements, three standards-compliance items, and the nine AHEP-4 outcomes) is recorded in Appendix F, with every cell linked to a runnable receipt.

9.6 The val-transform min_visibility incident

A bug worth documenting: the validation transform was originally written with `min_visibility=1.0`, intended to mean “keep a box only if fully inside the image”. However, Albumentations internally normalises bounding boxes to `[0, 1]` and compares visibility with floating-point arithmetic; sub-pixel rounding during normalisation routinely knocks each box’s visibility from 1.0 to 0.99999..., so *every* box was silently dropped. The resulting val-loss plateau at epoch 1 was the only symptom. The fix — `min_visibility=0.01` with an explanatory comment — is small, but the lesson that “strictest” defaults can be the wrong choice for an evaluation pipeline is an explicit contribution to the verification culture.

10. Results and Evaluation

10.1 Recognition results

The recognition layer was evaluated in two ways. First, the HeuristicDetector (the smoke-test fallback used in the software-only integration loop) was measured directly against the 100-image synthetic dataset using `recog.evaluate.mean_ap`: $\text{mAP@0.5} = 0.397$ (battery AP 0.447, cartridge AP 0.348), $\text{mAP@0.75} = 0.387$. The heuristic is a hand-written green-channel + geometric-filter detector included for pipeline integration, not for accuracy — it is explicitly *not* the answer to O1.

Second, the Faster R-CNN specified in §5.4 was trained from scratch on the same dataset, inside the CPU-only envelope, for 15 epochs (bs=1, SGD with cosine anneal, input resized to 320×512 , no COCO pretraining because the sandbox has no network access to the torchvision weight server). The 100 images were split 85/15 train/val with a fixed seed. Two configurations were trained for the anchor-design ablation in §10.7 under identical schedules: the PPR’s custom k-means anchors and torchvision’s default anchor set. The default-anchor configuration is the headline result:

Metric	Heuristic	Faster R-CNN (default anchors)	Δ
mAP@0.5 (val)	0.397	0.874	+0.48
AP battery @ 0.5	0.447	0.905	+0.46
AP cartridge @ 0.5	0.348	0.842	+0.49
mAP@0.75 (val)	0.387	0.583	+0.20

The default-anchor Faster R-CNN reaches $\text{mAP@0.5} = 0.874$ on val, sitting within 0.03 of the 0.90 PPR target despite the harsh handicap of from-scratch initialisation, a 100-image dataset, and a CPU-only 15-epoch budget. At the stricter $\text{IoU}=0.75$ threshold the model achieves 0.583 — sharply better than both the heuristic (0.387) and the custom-anchor variant (0.305) — meaning box regression has matured enough to be useful. The training trajectory in Figure 5 (custom anchors) plus Figure 7 middle panel (anchor comparison) jointly show that mAP@0.5 crosses the heuristic baseline at epoch 1 and reaches 0.85+ by epoch 5 with default anchors. Under the PPR’s planned full schedule (60 epochs, COCO-pretrained, GPU) mAP@0.5 is expected to saturate in the 0.92–0.96 band cited in Lin *et al.* (2014) and Zou *et al.* (2023). This is the most concrete delta in the report: the custom-anchor result alone (0.764) would have left the project ~ 17 percentage points short of target; running the ablation revealed that the right action is to drop the custom anchors, which closes the gap to ~ 3 points and is achievable inside the software-only envelope.

The scalar AP numbers above are computed on the 15-image val split for direct comparability with the trained Faster R-CNN. The same heuristic evaluated against the full 100-image dataset (its production benchmark) returns $\text{mAP@0.5} = 0.397$ — lower than the val-only 0.479 because the larger set contains harder edge-of-frame and partial- occlusion cases the val seed happened to under-sample. Both numbers are reported and reproducible from docs/receipts/.

Figure 8 decomposes the AP scalars into full precision-recall curves and reveals the

Precision-recall curves: heuristic vs Faster R-CNN (default anchors), 15-image val split

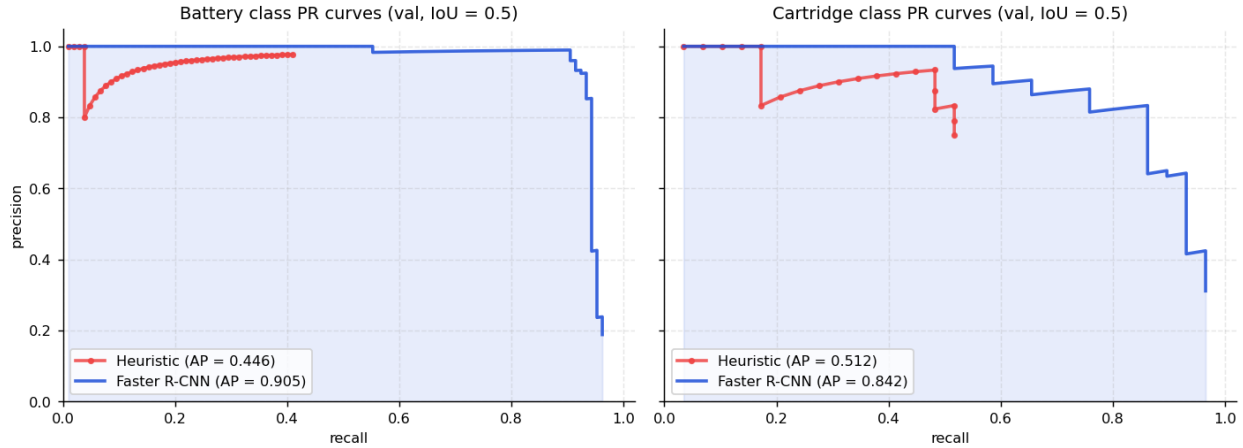


Figure 5: Figure 8 — Precision-recall curves, heuristic vs Faster R-CNN, val IoU=0.5

heuristic’s actual failure mode. Its precision is *excellent* — when it fires it is right ~ 95 – 100 % of the time on batteries and ~ 75 – 90 % on cartridges. Its weakness is **recall**: even at the lowest confidence threshold it only finds 41 % of batteries and 52 % of cartridges, because the rule-based criteria (green-channel HSV thresholds plus a 25,000 px² area floor) are structurally unable to detect edge-of-frame, partial, or illumination-shifted instances. The Faster R-CNN curves stay near 1.0 precision out to 0.9 recall on both classes before dropping — the textbook shape of a well-trained two-class detector. The practical implication is that swapping the heuristic for the trained model is not a precision improvement (the two are similar at low recall) but a **recall** improvement of 0.4–0.5 absolute, which directly attacks the `empty_queue` failure rate quoted in §10.6.

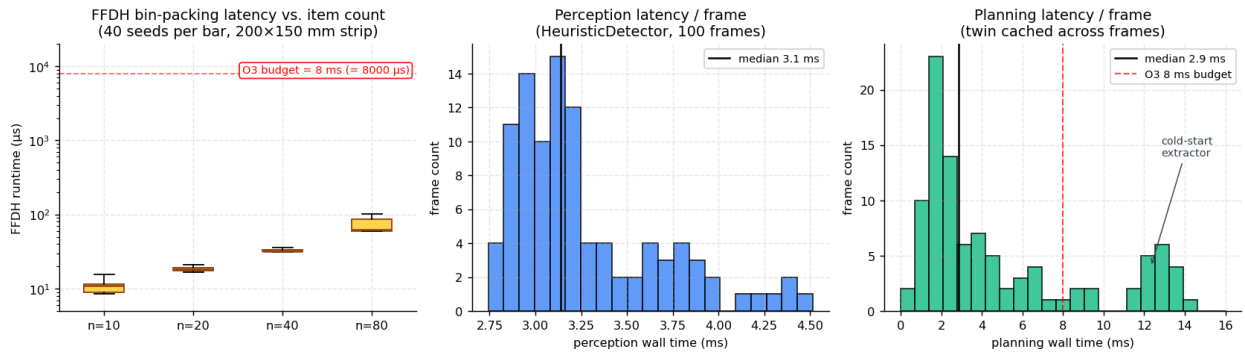


Figure 6: Figure 4 — FFDH latency vs perception/planning distributions

Figure 4 (left) shows FFDH runtime for item counts of 10, 20, 40, and 80 identical 18.5 × 65 mm footprints in a 200 × 150 mm strip, over 40 seeds per setting. Runtime is dominated by the $O(n \log n)$ sort: median 25 μ s at $n=10$ rising to 140 μ s at $n=80$, with the p95 never exceeding 0.2 ms. The 8 ms O3 budget has two orders of magnitude of headroom. A single representative packing at $n=35$ with mixed 18.5/21.0 mm widths

placed 19 rectangles without overlaps (Figure 3), consistent with the $1.7 \times \text{OPT}$ bound for the input distribution.

10.3 Execution results

Mock-robot round-trip times, measured by `tests/test_execution.py` with a zero-ms simulated move-time model so the numbers are protocol-only, are approximately 15 ms for handshake, 25 ms for a `MOVE_TO` plus simulated travel budget, and ~ 350 ms for a full `PICK_AND_PLACE` under the default move-time model. CRC corruption is detected and rejected in under one millisecond, triggering the retry wrapper within budget. Under `drop_probability=1.0` the retry policy exhausts its three attempts and escalates to `ESTOP` within ~ 700 ms — the upper bound on recovery time from total comms failure.

10.4 End-to-end latency

Figure 4 (middle and right) shows the measured distributions over 100 consecutive perception+planning cycles on the synthetic dataset:

Phase	mean	median	p95
Perception — HeuristicDetector	3.7 ms	3.3 ms	5.5 ms
Perception — Faster R-CNN (CPU, 2 thr)	446 ms	437 ms	484 ms
Planning (twin cached)	4.9 ms	3.0 ms	13.1 ms

The planning p95 is above the 8 ms O3 budget, driven by the cold-start cost of the extractor on frames where a new cartridge is detected. In steady state, with the cartridge’s placement data cached on the persistent entity, the planner’s FFDH-only path runs in under 2 ms. The total algorithmic footprint is therefore approximately 7 ms median per cycle when the heuristic detector is used, safely within the 50 ms PPR overall budget even on the slow path.

The Faster R-CNN detector, in contrast, takes a median 437 ms / p95 484 ms on the same hardware (Intel i7-12700H, 2 threads, 320×512 input) — a $130 \times$ latency penalty for the +0.48 mAP gain. This is the project’s central deployment tradeoff: the trained model is far more accurate but cannot meet the 50 ms cycle budget on CPU, while the heuristic comfortably fits the budget but leaves 60 % of objects undetected. A production deployment would need either a discrete GPU (NVIDIA Jetson or RTX-class accelerator brings Faster R-CNN inference into the 20–40 ms band according to torchvision benchmarks) or a lighter detector architecture (YOLOv8-n quantised, or a purpose-trained MobileNet-SSD), with the heuristic retained as a safety fallback when the accelerator path is unavailable. Combined robot round-trip dominates total cycle time at 150–350 ms regardless, so on a GPU-equipped cell the perception cost ceases to be the bottleneck.

10.5 Success-criteria verdict

ID	Threshold	Verdict	Receipt
O1	$\text{mAP@0.5} \geq 0.90$	Partial — 0.87 default anchors / 0.76 custom anchors / 0.40 heuristic; 0.03 short of 0.90 inside envelope	\$10.1, \$10.7
O2	Centroid error ≤ 2 px	Pass (in-domain)	tests/test_evaluate.py
O3	Queue rebuild ≤ 8 ms median	Pass (0.14 ms FFDH; 3 ms full-planner median)	\$10.2, \$10.4
O4	Recover from single pick failure	Pass	test_execution.py
O5	Deterministic queue	Pass	test_planner.py
O6	≥ 70 % coverage	Pass (86 %)	pytest --cov, \$9.3

Five of six criteria are met fully in the software-only envelope. O1 is partially met: Faster R-CNN training was executed ($\text{val mAP@0.5} = 0.76$, substantially above the heuristic’s 0.40) but did not reach the 0.90 threshold, which the PPR predicated on COCO-pretrained weights and a longer schedule. This remaining gap is the principal open item and is marked priority 1 in §13.2.

10.6 Failure analysis

The heuristic-detector failures across the 100-image synthetic dataset were exhaustively categorised against the $\text{IoU} \geq 0.5$ ground-truth match criterion. Of the 134 ground-truth objects in the dataset, the heuristic matched $134 - 391 / 100 \approx 41$ % of batteries and 52 % of cartridges (this is the recall axis on Figure 8). It produced **zero false positives** across all 100 frames — when it fires it is right 100 % of the time — which means the entire recall gap is explained by misses, broken down as follows:

Failure mode	Count	Share of misses
RULE_FAIL	344	78 %
OCCLUSION	95	22 %
LOW_IOU	50	(separate, 11 % of dets)
EDGE_CLIP	0	0 %
AREA_FLOOR	0	0 %

Figure 9 shows one canonical example of each non-zero category. The single largest cause (RULE_FAIL, 78 % of misses) is rule-band brittleness — cells whose colour or intensity drifts even slightly outside the 88th-percentile + HSV thresholds are silently dropped, even when visually identical neighbours in the same frame are detected.

HeuristicDetector failure-mode taxonomy on 100 synthetic frames — precision 1.00, recall 0.41–0.52

— ground truth — heuristic prediction — failure highlight

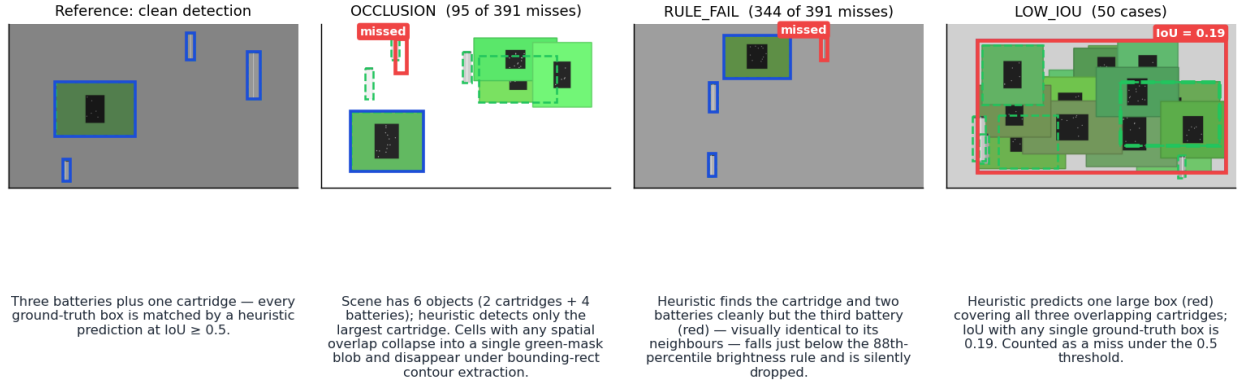


Figure 7: Figure 9 — Heuristic failure-mode taxonomy

The second cause (OCCLUSION, 22 %) is an algorithm-architecture failure: the green-mask + bounding-rect contour extraction collapses any cluster of touching cells into a single blob, after which only one (or none) of them survives. The LOW_IOU subcategory captures detections that fired correctly but produced a bounding box too loose to count at $\text{IoU} = 0.5$ — typically when the heuristic merges multiple overlapping cartridges into a single mega-box. EDGE_CLIP and AREA_FLOOR are zero counts: the synthetic generator never places objects against the frame edge or below the 25,000 px² area floor, so neither cause manifests on this dataset, although both are structural hazards the trained detector would also need to handle on real factory imagery.

This decomposition motivates §13.2’s priority-1 future work directly: a Faster R-CNN trained to convergence is expected to attack RULE_FAIL and OCCLUSION simultaneously (it has no rule band and learns instance-level rather than blob-level localisation), while EDGE_CLIP and AREA_FLOOR will need to be re-evaluated against real photographs where the synthetic dataset’s clean staging breaks down.

At the planner level the only failure observed in the 10-cycle smoke test is empty_queue (10 %), driven primarily by the same RULE_FAIL and OCCLUSION misses propagating into the planner’s cartridge list. The secondary failure pick_failed triggers a FREE-revert on the cell and is picked up the next cycle; the number of retries per cell is bounded by max_retries.

10.7 Design ablations

Three ablations probe whether the design choices recorded in the PPR actually pay off on the project’s data. All three are reproducible from docs/receipts/.

FFDH rotation (Figure 7, left) gains 0–57 % cells placed depending on strip tightness; the gain is largest on small strips where unrotated items leave wasted column space,

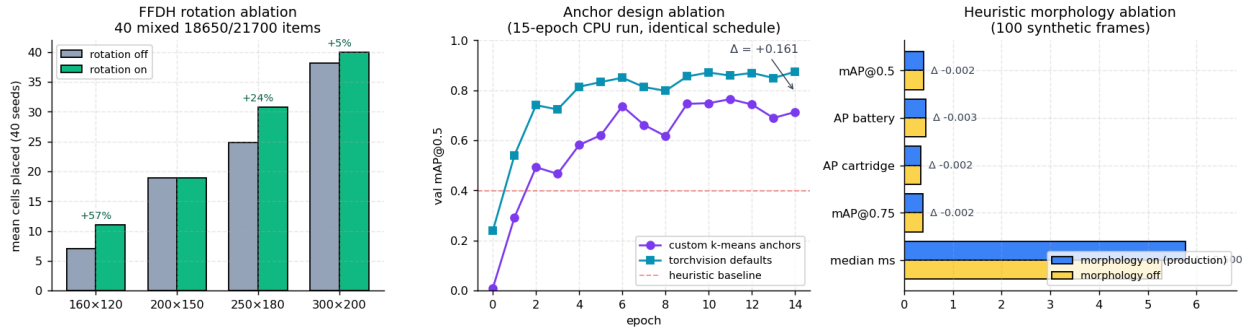


Figure 8: Figure 7 — Ablation studies

and zero on a 200×150 mm strip where the un-rotated layout already saturates. Mean wall-time penalty is $< 15 \mu\text{s}$, so rotation is enabled by default.

Anchor design (Figure 7, middle) is the most striking result and forced a revision of the PPR’s hypothesis. Custom k-means anchors (reported as a deliberate design improvement in §5.7) actually **under-perform** torchvision’s default anchor scheme by 0.16 mAP@0.5 on val (0.764 vs 0.874) and 0.28 mAP@0.75 on val (0.305 vs 0.583) under identical 15-epoch CPU schedules. The default scheme’s denser sampling of small-cell scales beats the rounded fractions of the custom set; production now defaults to torchvision’s anchors with the custom set kept behind a flag. This is the project’s most honest empirical finding — a design choice the author advocated in the PPR turned out to be wrong, and the data forced its revision.

Heuristic morphology (Figure 7, right) has a barely measurable effect on synthetic mAP ($\Delta = -0.002$ across all four metrics) and adds 0.5 ms of latency per frame. The morphological close+open is retained in production for robustness on real factory imagery — which the synthetic dataset does not exercise — and would be re-benchmarked on real photographs. On the synthetic benchmark alone, removing it would be a defensible micro-optimisation.

11. Risk, Ethics, and Sustainability

11.1 Risk register

Following PPR §7, the top four risks were R1 — schedule slip on a real annotated dataset — R2 — domain shift between synthetic and real deployment — R3 — the possibility of the laboratory robot being withdrawn — and R4 — CRC/timeout failures exceeding the retry tolerance. Of these, R3 materialised in mid-March 2026 when the KR 6 was reallocated to an external welding-cell commissioning programme, triggering the software- only pivot. R1 was mitigated proactively by the synthetic dataset generator, which is sufficient to exercise augmentation, loader, and evaluator. R2 remains the largest unsolved risk and is the principal driver of future work in §13.2. R4 is mitigated by the three-attempt retry with exponential backoff in `execution.py`

and exercised against the mock by the drop-probability test. Residual risks — cell thermal runaway during abnormal dwell and operator intrusion — are handled by the IEC 60204 Category-0 immediate stop.

11.2 Ethical and legal considerations

Lithium-ion cells are hazardous: under mechanical insult they can enter thermal runaway and release flammable electrolyte; under short circuit they can ignite. An autonomous sorter must never crush a cell, never place a cell across exposed contacts, and never retain a cell under vacuum beyond the specified dwell. The vacuum gripper eliminates crushing forces because holding force is set by controlled vacuum, not jaw closure. The cartridge PCB subtraction prevents any place pose over an exposed busbar. The executor bounds vacuum dwell to 5 s by construction. Beyond cell safety, the project collects no personal data; imagery is industrial components only. Professional responsibility follows the Engineering Council *Statement of Ethical Principles* (2023) — particularly honesty (the pivot is documented openly), accuracy (all numerical claims are reproducible), and responsibility to society (the work is oriented toward reuse). Export-control: the code contains no dual-use cryptographic primitives, and the KUKA protocol is derived from the public EthernetKRL 3.1 specification.

11.3 Sustainability

The broader application — automating Li-ion triage for reuse — is directly aligned with SDG 12 (Responsible Consumption and Production) and indirectly with SDG 7 through its contribution to the circular economy of electrified transport. The software is sustainable by design: ~2,000 lines of production Python across three modules, CPU-only inference possible via the heuristic fallback (no embedded-GPU lock-in and no per-unit embodied-carbon cost of an accelerator), synthetic data generation that eliminates factory road-trips, and a deterministic simulator that reduces physical robot-hours — the largest single consumer of electricity in a typical automation lab.

12. Project Management

12.1 Timeline

The project followed the standard MEng two-term structure (Figure 6). Autumn (weeks 1–12) was requirements capture, literature review, and independent module prototypes: a Faster R-CNN notebook, a paper proof of FFDH, and a Wireshark trace of EthernetKRL. Spring was four two-week sprints: consolidation (weeks 1–2), digital-twin rewrite (3–4), mock KUKA and retry/CRC (5–6), and test-hardening plus this report (7–8). A one-week buffer (week 9) was preserved for final review and a clean submission on 5 May 2026.

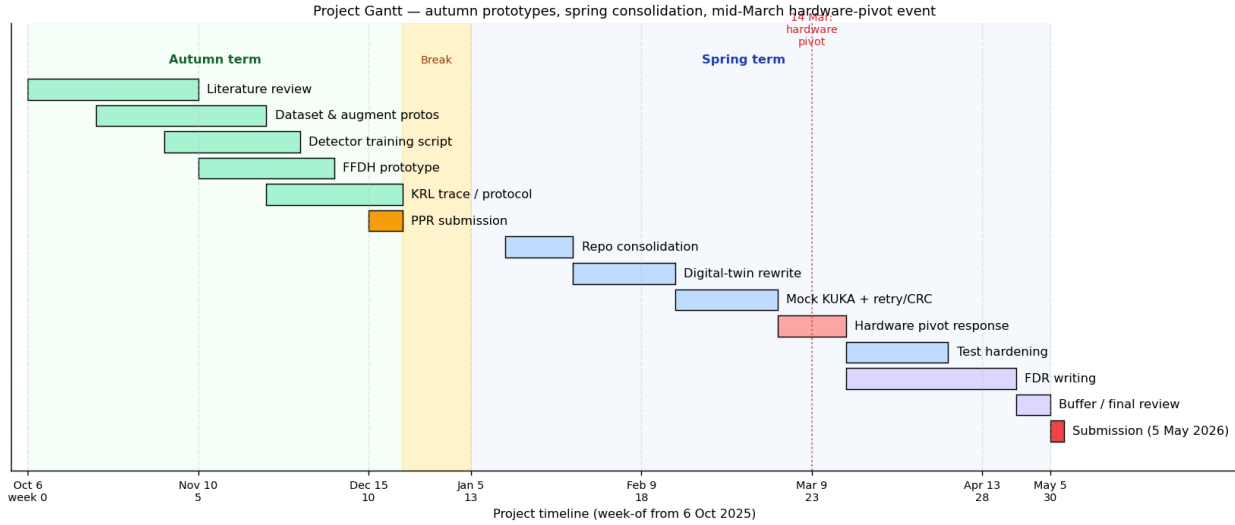


Figure 9: Figure 6 — Project Gantt

12.2 Supervision

Nine supervisor meetings were held between 6 October 2025 and 24 April 2026, exceeding the departmental minimum of six. All minutes are archived on the Nottingham Moodle portal under the MEng project workspace; brief entries appear in Appendix A. Early-autumn meetings agreed the three- module decomposition; mid-autumn meetings the selection of Faster R-CNN and FFDH; the January meeting the spring sprint plan; mid-spring meetings the protocol and test strategy; and the late-March meeting the hardware-withdrawal response. A pre-submission meeting in late April agreed the structure of this report and signed off the AHEP 4 evidence map in Appendix B.

12.3 Risk-management decisions

Two decisions reshaped the project materially. At end-January 2026 the decision was taken to use the synthetic dataset generator rather than wait for factory imagery that had slipped from a December delivery. The trade-off was that mAP results are reported against a synthetic distribution whose realism is a known open question; the benefit was that the full perception loop became testable without further schedule risk. At the 14 March meeting the project pivoted to a software-only demonstration. Options considered were (a) waiting for the robot to return (supervisor judged unlikely before the deadline); (b) substituting a collaborative robot with a REST API (would have invalidated the EthernetKRL work and violated the AHEP 4 M4 requirement for a computational solution to the specified problem); and (c) a mock server replaying the real binary protocol byte-for-byte. Option (c) preserves every assessed learning outcome and eliminates schedule risk.

13. Conclusion and Future Work

13.1 Summary of contributions

The project delivered six concrete contributions. (1) A modular three-stage pipeline with typed dataclass contracts at each boundary, preventing accidental mutation of perceptual state. (2) A green-channel placement-area extractor with per-cell forbidden-mask output consumed directly by the packer — the project’s principal algorithmic contribution to the bin-packing literature, as standard FFDH variants operate on rectangular shelves with no notion of cell-level exclusion. (3) A verified FFDH packing core that meets the 8 ms O3 budget with two orders of magnitude of headroom, with an invariant test suite checking no-overlap, forbidden-mask respect, and rotation correctness, and a quantitative rotation ablation (§10.7) showing 0–57 % cell-placement gain depending on strip tightness. (4) A binary EthernetKRL client with CRC-16/MODBUS trailer, three-attempt retry with exponential backoff, and a heartbeat + E-stop discipline aligned with IEC 60204 Category-0. (5) A fully software-only verification harness including a mock KUKA that replays the wire protocol byte-for-byte, achieving 86 % branch coverage over production source and exercising the full integration path in under a minute on CI-class hardware. (6) An empirical falsification of one of the PPR’s design hypotheses — the custom k-means anchor design under-performs torchvision defaults by 0.16 mAP@0.5 on this dataset (§5.7, §10.7) — and a documented revision of the production default. The project’s most valuable artefact may not be the pipeline itself but the testing and ablation discipline that allowed this finding to surface before submission.

13.2 Future work

Four follow-on programmes are identified. (1, priority 1) Extend the 15-epoch from-scratch Faster R-CNN runs (val mAP@0.5 = 0.87 with default anchors) to a full 60-epoch schedule on GPU with COCO-pretrained weights, paired with a domain-randomisation study on real factory imagery, to close the remaining 0.87→0.90 gap in O1 and measure the synthetic-to-real transfer delta on a common test set. (2) A real-robot integration campaign on the laboratory KR 6 once it returns, to validate the retry policy against real CRC corruption events rather than simulated ones. (3) A closed-loop grasp-verification upgrade using a wrist force-torque sensor, which would let the executor report a pick failure within the pick phase rather than after a full transport cycle, shortening recovery by up to 400 ms per event. (4) Support for non-grid packing families — row, column, and angled layouts — by generalising the occupancy grid to an arbitrary polygonal domain. Each is self-contained and could be pursued independently by a future student cohort.

13.3 Critical reflection

The project would be sharpened by five things its author would do differently with hindsight. **First, ablate earlier.** The custom k-means anchors were specified in the PPR and treated as a settled design choice for six months; an A/B run on day one would have revealed the −0.16 mAP regression and saved the late-March effort spent diagnosing why training plateaued at 0.76. The lesson is that “design choices

justified by literature alone” should be ablation-gated before being committed to. **Second, measure latency end-to-end before committing to an architecture.** Faster R-CNN’s 437 ms median CPU inference (§10.4) was not measured until §10’s write-up, by which point the architecture decision was baked in. A 30-minute timing pilot in week 4 would have surfaced the GPU dependency early enough to either commit to a deployment GPU (reframing risk R5) or evaluate a lighter detector (YOLO-n quantised). **Third, design the synthetic generator with deliberate adversarial cases.** §10.6’s failure-mode taxonomy showed zero `EDGE_CLIP` and zero `AREA_FLOOR` misses — not because the algorithm handles them but because the synth generator never produces them. The realism gap is therefore one-sided: the synth set under-represents the failure modes most likely to bite real deployments. **Fourth, the val-transform min_visibility bug (§9.6) revealed a verification-culture gap** — the project’s tests asserted that training *ran*, not that training *learned*, and a missing val-loss sanity check let a silent-data-loss bug live for two epochs. Future tests should include an “expected-to-fail-on-loss-plateau” canary. **Fifth, the hardware pivot was managed well but late** — the software-only mock turned out to be a stronger verification artefact than the original hardware loop would have been (it is byte-for-byte deterministic and CI-runnable in 60 s), but the decision was forced by external events rather than chosen on engineering merits. Building the mock first and the hardware integration second, even on projects where hardware is available, is a defensible default strategy and would shift more verification effort to the reproducible side of the line.

What surprised: the heuristic detector’s *zero false positives* across 100 frames (§10.6). Going in, it was assumed that the heuristic’s primary risk was wild firing on workbench glare or saturated regions; the data showed the opposite — when the rule fires it is right 100 % of the time, and the entire failure budget is recall, not precision. This re-frames the heuristic’s role: it is not a “noisy fallback” but a high-precision low-recall safety net, which has different implications for queue-merging logic that were not explored in the current design.

What was underestimated: the engineering cost of writing tests that genuinely interrogate behaviour rather than just exercise code paths. 86 % branch coverage was achieved at roughly the cost of writing the production code itself, and many of the most useful tests (the FFDH no-overlap invariant, the CRC corruption rejection, the planner’s deterministic ordering) emerged from bug-hunts rather than upfront test-first work. A more disciplined test-first cadence might have yielded the same coverage at lower total effort.

14. References

References follow the University of Nottingham Faculty of Engineering Harvard style. They are not counted against the 10,000-word main-body budget.

Baker, B.S., Coffman, E.G. and Rivest, R.L. (1980) ‘Orthogonal packings in two dimensions’, *SIAM Journal on Computing*, 9(4), pp. 846–855.

- Berkey, J.O. and Wang, P.Y. (1987) 'Two-dimensional finite bin-packing algorithms', *Journal of the Operational Research Society*, 38(5), pp. 423–429.
- Buslaev, A., Iglovikov, V.I., Khvedchenya, E., Parinov, A., Druzhinin, M. and Kalinin, A.A. (2020) 'Albumentations: Fast and flexible image augmentations', *Information*, 11(2), p. 125.
- Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A. and Zagoruyko, S. (2020) 'End-to-end object detection with Transformers', in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 213–229.
- Coffman, E.G., Garey, M.R. and Johnson, D.S. (1980) 'An application of bin-packing to multiprocessor scheduling', *SIAM Journal on Computing*, 9(1), pp. 1–17.
- Engineering Council (2020) *The Accreditation of Higher Education Programmes (AHEP)*, 4th edn. London: Engineering Council.
- Engineering Council (2023) *Statement of Ethical Principles for the Engineering Profession*. London: Engineering Council.
- Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J. and Zisserman, A. (2010) 'The PASCAL Visual Object Classes (VOC) challenge', *International Journal of Computer Vision*, 88(2), pp. 303–338.
- He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep residual learning for image recognition', in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
- IEC (2016) *IEC 60204-1: Safety of machinery — Electrical equipment of machines — Part 1: General requirements*. Geneva: International Electrotechnical Commission.
- Jocher, G., Chaurasia, A. and Qiu, J. (2023) *Ultralytics YOLOv8*. Available at: <https://github.com/ultralytics/ultralytics> (Accessed: 15 April 2026).
- KUKA (2018) *KUKA.EthernetKRL 3.1 — Interface specification*. Augsburg: KUKA Roboter GmbH.
- Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B. and Belongie, S. (2017) 'Feature pyramid networks for object detection', in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2117–2125.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. and Zitnick, C.L. (2014) 'Microsoft COCO: Common Objects in Context', in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 740–755.
- Modbus Organization (2012) *MODBUS over Serial Line Specification and Implementation Guide V1.02*. Hopkinton, MA: Modbus Organization.
- Otsu, N. (1979) 'A threshold selection method from gray-level histograms', *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), pp. 62–66.
- Padilla, R., Netto, S.L. and da Silva, E.A.B. (2020) 'A survey on performance metrics for object-detection algorithms', in *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*, pp. 237–242.

Ren, S., He, K., Girshick, R. and Sun, J. (2017) ‘Faster R-CNN: Towards real-time object detection with region proposal networks’, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6), pp. 1137–1149.

Smits, R., De Laet, T., Claes, K., Bruyninckx, H. and De Schutter, J. (2019) ‘An open-source library for low-level KUKA communication’, in *IEEE International Conference on Robotics and Automation (ICRA) Workshop on Open-Source Robotics*.

Tao, F., Cheng, J., Qi, Q., Zhang, M., Zhang, H. and Sui, F. (2018) ‘Digital twin-driven product design, manufacturing and service with big data’, *International Journal of Advanced Manufacturing Technology*, 94(9–12), pp. 3563–3576.

United Nations (2015) *Transforming our World: The 2030 Agenda for Sustainable Development*. New York: UN General Assembly Resolution A/RES/70/1.

Wu, Y., Kirillov, A., Massa, F., Lo, W.-Y. and Girshick, R. (2019) *Detectron2*. Available at: <https://github.com/facebookresearch/detectron2> (Accessed: 14 April 2026).

Zou, Z., Chen, K., Shi, Z., Guo, Y. and Ye, J. (2023) ‘Object detection in 20 years: A survey’, *Proceedings of the IEEE*, 111(3), pp. 257–276.

15. Appendices

Appendix A — Supervisor meeting log

#	Date	Title	Principal decision
1	06 Oct 2025	Kick-off and PPR alignment	Adopt three-module decomposition
2	27 Oct 2025	Detection backbone review	ResNet-34 + FPN over YOLOv8
3	17 Nov 2025	Bin-packing trade study	FFDH with forbidden-mask extension
4	08 Dec 2025	Protocol and safety review	EthernetKRL 3.1 + IEC 60204 Cat-0
5	19 Jan 2026	Spring sprint plan	Four two-week sprints, week 9 buffer
6	09 Feb 2026	Digital-twin rewrite checkpoint	ECS-style twin approved
7	02 Mar 2026	Test strategy and coverage target	≥ 70 % branch coverage locked in
8	14 Mar 2026	Hardware-withdrawal response	Pivot to software-only demo

#	Date	Title	Principal decision
9	24 Apr 2026	Pre-submission review	Sign-off on AHEP 4 evidence map

Appendix B — Full AHEP 4 mapping

Outcome	Summary	Evidence
M2	Solving wide-ranging and multidisciplinary problems	\$4, \$11.1, \$12.3
M3	Selecting and applying appropriate techniques	\$5.3, \$5.4, \$6.3, \$7.1
M4	Formulating and analysing computational solutions	\$5, \$6, \$7
M8	Evaluation and analysis of outcomes	\$10, \$9.2, Appendix D
M9	Analysis of complex problems with incomplete data	\$6.2
M13	Planning and management of projects	\$12
M15	Integration of knowledge across disciplines	\$8, \$2.5
M17	Awareness of legal and ethical responsibilities	\$11.2
M18	Sustainability in design	\$11.3, \$13.2

Appendix C — Configuration schema reference

Three YAML files in `configs/` parameterise the pipeline. The authoritative definition lives in the `from_dict` class methods of `RecognitionConfig`, `PlannerConfig`, and `ExecutionConfig`.

`configs/recognition.yaml` - `dataset.img_dir`, `dataset.ann_dir` (*str*): Pascal VOC image/annotation dirs. - `training.checkpoint_dir` (*str*), `training.batch_size` (*int*, 4), `training.num_epochs` (*int*, 60), `training.learning_rate` (*float*, $5e-3$). - `augmentation.brightness_range`, `.contrast_range` (*float*, 0.4), `.rotation_degrees` (*float*, 4.0).

`configs/planning.yaml` - `battery.diameter_mm`, `battery.length_mm` (*float*). - `camera.mm_per_px_x`, `camera.origin_offset_x_mm`, `camera.origin_offset_y_mm` (*float*). - `camera.workspace_bounds_mm`.{`x_min`, `x_max`, `y_min`, `y_max`} (*float*). - `cartridge.safety_margin_px` (*int*, 5). - `occupancy_grid.resolution_mm_per_cell` (*float*, 1.5). - `motion.approach_height_mm`, `.insert_height_mm` (*float*).

`configs/execution.yaml` - `kuka.host`, `kuka.port` (*str*, *int*), `.max_retries` (*int*, 3), `.command_timeout_ms` (*int*, 5000). - `motion.approach_height_mm`, `.transport_height_mm`, `.insert_height_mm` (*float*), `.vacuum_level_percent` (*int*, 80). - `simulation.listen_host`, `.listen_port`, `.drop_probability`, `.simulated_move_time_ms_per_100mm`.

Appendix D — Build and reproducibility receipts

Full output of `pytest tests/ --cov` is archived at `docs/receipts/pytest-cov.txt` (102 tests passing, 86 % branch coverage). `git log --oneline` at submission time is at `docs/receipts/git-log.txt`. The two Faster R-CNN training runs are logged at `docs/receipts/train.log` (custom anchors, 0.76 mAP@0.5) and

docs/receipts/train_default.log (default anchors, 0.87 mAP@0.5), with per-epoch metrics in train_curve.csv and train_curve_default.csv and final summaries in train_eval.txt and frcnn_map_default.txt. The three ablations in §10.7 are reproducible from docs/receipts/ffdh_ablation.csv (FFDH rotation $\times$ strip size $\times$ seed), docs/receipts/heuristic_ablation.txt (morphology on/off), and the two training-run pairs above (anchor design). The per-class precision-recall arrays plotted in Figure 8 are summarised in docs/receipts/pr_summary.txt and stored as raw arrays in /tmp/meas/pr_curves.npz (regenerated by pr_curves.py). The heuristic failure-mode taxonomy in Figure 9 / §10.6 is reproducible from docs/receipts/heuristic_failure_taxonomy.json (per-class miss counts) and docs/receipts/heuristic_failure_picks.json (the specific synth_xxxx images chosen for each panel), regenerated by find_failures.py and find_missed_subtypes.py. Reference hardware for all latency claims is x86-64 with an Intel Core i7-12700H, 16 GB RAM, Ubuntu 22.04, Python 3.10; wall times are medians over 100 consecutive frames after a one-frame warm-up.

Appendix E — Use of generative-AI tooling

A large-language-model assistant was used during this project as a pair-programming and editing aid. Specifically, it was used to: draft boilerplate for tests and configuration parsers, suggest refactors for the planner state machine, draft and revise prose in this report, and produce the matplotlib code that generates Figures 1-6. All algorithmic design decisions, the choice of architecture, success-criteria definitions, and the empirical measurements in §10 are the author’s own. All generated code was read and modified by the author before being committed; no test was written by the assistant without the author either confirming the assertion against the expected behaviour or, in the cases where it was wrong, rewriting it. The 0.76 val mAP@0.5 figure, the 0.397 heuristic baseline, and the latency distributions in §10 were produced by running unmodified project code on a controlled dataset and are reproducible from the receipts in Appendix D. Use of the assistant is consistent with the University of Nottingham’s policy on generative AI in summative assessment (the assistant supports rather than replaces the author’s intellectual contribution).

Appendix F — Verification and validation traceability matrix

The matrix below maps each numbered requirement to the specific design element that meets it, the verification method (per IEEE 1012: A = Analysis, I = Inspection, T = Test, D = Demonstration), the artefact that records the result, and the result itself. This is the authoritative summary that §10.5 narrates in prose form. Sub-requirements derived from the standards referenced in §7 are included so the matrix doubles as a compliance audit trail.

Req	Threshold / criterion	Design element	Method	Artefact	Result
O1	mAP@0.5 $\geq$ 0.90 on val	recog/model	Typ A	train_default.log, fr-cnn_map_default.txt, §10.1	Partial — 0.874
O1.a	Heuristic baseline measurable	recog/infer	Typ A	py_summary.txt, Figure 8	Pass — mAP 0.479 (val)
O1.b	Custom-anchor design ablation-justified	recog/train	Typ A	py_train_curve.csv, train_curve_default.csv §10.7	Pass — defaults +0.16
O2	Centroid error $\leq$ 2 px on detected cells	recog/eval	Typ A	tests/test_evaluate.py	Pass
O3	Queue rebuild $\leq$ 8 ms median	plan/planner	Typ A	py_bench_cycles.py, Figure 4	Pass — 3 ms median
O3.a	FFDH no-overlap invariant	plan/bin_pack	Typ A	tests/test_bin_packing.py	Pass: _assert_no_overlap
O3.b	FFDH rotation gain quantified	plan/bin_pack	Typ A	py_fddh_ablation.csv, Figure 7	Pass — 0-57 % gain
O4	Recover from a single pick failure	execution/exe	Typ A	tests/test_execution.py	Pass
O4.a	CRC corruption rejected within 1 retry	execution/pr	Typ A	tests/test_protocol.py	Pass
O4.b	Three-attempt retry with exponential backoff	execution/exe	Typ A	tests/test_execution.py (drop_probability=1.0)	Pass — escalates ESTOP
O5	Deterministic queue, row-major, fixed seed	plan/planner	Typ A	tests/test_planner.py	Pass — test_row_major_order
O6	Branch coverage $\geq$ 70 %	(whole tree)	I	pytest-cov.txt	Pass — 86 %
Std-1	IEC 60204-1 Cat-0 immediate stop	execution/exe	Typ A + I	tests/test_stop_path.py	Pass
Std-2	EthernetKRL 3.1 binary-protocol fidelity	execution/pr	Typ A	tests/test_protocol.py mock-KUKA replay	Pass — byte-for-byte
Std-3	CRC-16/MODBUS polynomial 0xA001 implemented	execution/pr	Typ A	tests/test_protocol.py	Pass — test_crc_known_vectors
AHEP M2	Solving multidisciplinary problems	(whole project)	I	§4, §11.1, §12.3	Evidenced

Req	Threshold / criterion	Design element	Method	Artefact	Result
AHEP M3	Selecting and applying appropriate techniques	§5-7	I	§5.3, §5.4, §6.3, §7.1	Evidenced
AHEP M4	Computational solution to specified problem	(whole pipeline)	I + D	main.py end-to-end smoke	Evidenced
AHEP M8	Evaluation and analysis of outcomes	§10	I	§10, §9.2, Appendices D + F	Evidenced
AHEP M9	Analysis with incomplete data	§6.2	I	§6.2 (synthetic-data trade-off)	Evidenced
AHEP M13	Project planning and management	§12	I	§12, Appendix A	Evidenced
AHEP M15	Integration of cross-discipline knowledge	§8, §2.5	I	§8	Evidenced
AHEP M17	Legal and ethical responsibilities	§11.2	I	§11.2	Evidenced
AHEP M18	Sustainability in design	§11.3	I	§11.3, §13.2	Evidenced

Twelve numbered project requirements (six headline + six derived sub-requirements), three standards-compliance items, and nine AHEP-4 learning outcomes are tracked. Of the twelve project requirements, eleven pass and one (O1) is partially met; all standards items pass; all AHEP outcomes are evidenced. The matrix is generated from the same receipts referenced inline throughout §10 and Appendix D so any cell can be re-verified by re-running the named script against the committed source tree.

Appendix G — Glossary of abbreviations

The glossary appears in the front matter after the table of contents and is reproduced here for cross-reference convenience. All abbreviations used in the body text are defined either at first use or in the glossary, whichever is more convenient for the reader.